step 1:Install and import libraries

```
!pip install nltk pandas

import pandas as pd
import nltk
import re
from collections import defaultdict, Counter

nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.1)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger.zip.
True
```

step 2:Load dataset

```
from google.colab import files
uploaded = files.upload()

df = pd.read_csv(list(uploaded.keys())[0])
df.head()
```

Choose files Twitter_Data.csv

Twitter_Data.csv(text/csv) - 20895533 bytes, last modified: 29/01/2026 - 100% done

```
Saving Twitter_Data.csv to Twitter_Data.csv
```

	clean_text	category
0	when modi promised “minimum government maximum...	-1.0
1	talk all the nonsense and continue all the dra...	0.0
2	what did just say vote for modi welcome bjp t...	1.0
3	asking his supporters prefix chowkidar their n...	1.0
4	answer who among these the most powerful world...	1.0

step 3:Preprocess Tweets (Remove Noise)

```
def clean_tweet(text):
    text = re.sub(r"http\S+", "", text)     # remove URLs
    text = re.sub(r"@\w+", "", text)        # remove mentions
    text = re.sub(r"#\w+", "", text)        # optional: remove hashtags
    text = re.sub(r"[^A-Za-z\s]", "", text) # remove punctuation/emojis
    return text.lower()

df['clean_text'] = df['clean_text'].astype(str).apply(clean_tweet)
df.head()
```

	clean_text	category
0	when modi promised minimum government maximum ...	-1.0
1	talk all the nonsense and continue all the dra...	0.0
2	what did just say vote for modi welcome bjp t...	1.0
3	asking his supporters prefix chowkidar their n...	1.0
4	answer who among these the most powerful world...	1.0

step 4:Tokenize & POS Tag with NLTK

```
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')

def pos_tag_tweet(text):
    tokens = nltk.word_tokenize(text)
    return nltk.pos_tag(tokens)

df['pos_tags'] = df['clean_text'].apply(pos_tag_tweet)
df[['clean_text', 'pos_tags']].head()
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
```

	clean_text	pos_tags
0	when modi promised minimum government maximum ...	[(when, WRB), (modi, NN), (promised, VBD), (mi...
1	talk all the nonsense and continue all the dra...	[(talk, NN), (all, PDT), (the, DT), (nonsense,...
2	what did just say vote for modi welcome bjp t...	[(what, WP), (did, VBD), (just, RB), (say, VB)...
3	asking his supporters prefix chowkidar their n...	[(asking, VBG), (his, PRP$), (supporters, NNS)...
4	answer who among these the most powerful world...	[(answer, NN), (who, WP), (among, IN), (these,...

step 5:Build HMM Parameters

```
transition_counts = defaultdict(Counter)
emission_counts = defaultdict(Counter)
tag_counts = Counter()

for tweet in df['pos_tags']:
    prev_tag = "<START>"
    for word, tag in tweet:
        transition_counts[prev_tag][tag] += 1
        emission_counts[tag][word] += 1
        tag_counts[tag] += 1
        prev_tag = tag
transition_probs = defaultdict(dict)
emission_probs = defaultdict(dict)

for prev_tag, next_tags in transition_counts.items():
    total = sum(next_tags.values())
    for tag, count in next_tags.items():
        transition_probs[prev_tag][tag] = count / total

for tag, words in emission_counts.items():
    total = sum(words.values())
    for word, count in words.items():
        emission_probs[tag][word] = count / total
```

step 6:Analyze HMM Issues

```
for prev_tag in list(transition_probs)[:5]:
    print(prev_tag, transition_probs[prev_tag])
```

```
<START> {'WRB': 0.031275119191522524, 'NN': 0.3365526805053598, 'WP': 0.017653230903278458, 'VBG': 0.020113761175163985, 'JJ': 0
WRB {'NN': 0.2648639790232711, 'VBZ': 0.028515240904621434, 'JJS': 0.0008194034742707309, 'JJ': 0.1857423795476893, 'VBN': 0.016
NN {'VBD': 0.06108985785688273, 'JJ': 0.05632281494560407, 'VBG': 0.027593150701921136, 'WRB': 0.00918005283434977, 'NN': 0.3647
VBD {'JJ': 0.18980089625049634, 'PRP': 0.03621873050087923, 'RB': 0.09938170060695445, 'WP': 0.005672471495830733, 'NN': 0.16049
JJ {'NN': 0.5737809648332075, 'JJ': 0.10125848986630068, 'WP': 0.001835354117771263, 'VB': 0.009512440073522446, 'NNS': 0.155688
```

2. Rare & Unknown Tokens

```
rare_words = [word for tag in emission_counts
              for word, count in emission_counts[tag].items()
              if count == 1]

print("Rare word sample:", rare_words[:20])
```

```
Rare word sample: ['mover', 'sincere', 'wld', 'wasn', 'whatsapp', 'won', 'withot', 'wali', 'wicket', 'jaiwere', 'wht', 'workerst
```

step 7:Manual Viterbi Decoding (One Tweet)

```
test_tweet = "love this phone"
tokens = nltk.word_tokenize(test_tweet)
tags = list(tag_counts.keys())
V = [{}]
path = {}

for tag in tags:
    V[0][tag] = transition_probs["<START>"].get(tag, 1e-6) * \
                emission_probs[tag].get(tokens[0], 1e-6)
    path[tag] = [tag]

for t in range(1, len(tokens)):
    V.append({})
    new_path = {}

    for curr_tag in tags:
        prob_tag_pairs = []
        for prev_tag in tags:
            prob = V[t-1][prev_tag] * \
                   transition_probs[prev_tag].get(curr_tag, 1e-6) * \
                   emission_probs[curr_tag].get(tokens[t], 1e-6)
            prob_tag_pairs.append((prob, prev_tag))

        best_prob, best_prev = max(prob_tag_pairs)
        V[t][curr_tag] = best_prob
        new_path[curr_tag] = path[best_prev] + [curr_tag]

    path = new_path

best_final_tag = max(V[-1], key=V[-1].get)
print("Best Tag Sequence:", path[best_final_tag])
```

```
Best Tag Sequence: ['VB', 'DT', 'NN']
```

Double-click (or enter) to edit

```
Start coding or generate with AI.
```